Facial Attendance System

A Real-Time Attendance Marking System using OpenCV and the face_recognition Library

Project Report

Course: Artificial intelligence
University of Karachi / UBIT (BSCS)

Member	Name	Responsibility
1	Serena Salman	Face Registration, Encoding & Recognition Engine
2	Lamia Gul	Attendance Logic & Main Application

1.Introduction

Manual attendance tracking in classrooms and workplaces is time-consuming and prone to proxy attendance. This project implements a Facial Attendance System that automatically identifies registered individuals through a live webcam feed and logs their presence with a timestamp, removing the need for manual roll calls or sign-in sheets.

The system uses OpenCV for camera access and face detection, and the face_recognition library (built on dlib's deep learning face embedding model) to identify which registered person a detected face belongs to. Once a face is recognized, the system automatically records the person's name, the current date, and the current time into a CSV file, while preventing the same person from being marked more than once on the same day

1.1 Project Objectives

- Detect human faces in a live video stream using OpenCV.
- Recognize previously registered faces using the face_recognition library.

- Automatically mark attendance for each recognized person, without manual intervention.
- Store attendance records in a structured CSV file containing Name, Date, and Time.
- Prevent duplicate attendance entries for the same person on the same day

2. Tools and Technologies Used

Tool/ Library	Purpose
Python 3.11	Core programming language for the entire project
OpenCV (opencv-python)	Webcam access, frame capture, drawing bounding boxes and labels
face_recognition	Face detection and generation of 128-dimensional face encodings
dlib (via dlib-bin)	Underlying deep learning model used internally by face_recognition
NumPy	Numerical operations on face encoding arrays
pickle (standard library)	Saving and loading known face encodings to/from disk
csv (standard library)	Reading and writing the Attendance.csv file
datetime (standard library)	Generating the current date and time for each attendance record

3. System Workflow

The system operates in three sequential stages:

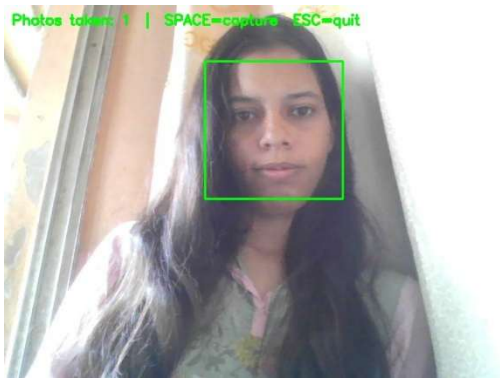
Stage 1 — Registration: Each person's face is captured through the webcam (register_faces.py) and saved as image files, organized in a folder per person under known_faces/.

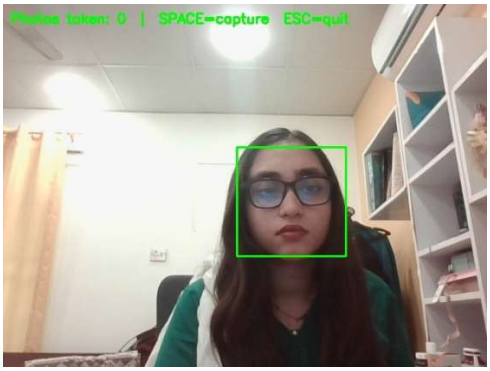
Stage 2 — Encoding: Every registered image is converted into a 128-dimensional numeric face encoding (encode_faces.py) and stored together with the person's name in encodings.pickle.

Stage 3 — Recognition & Attendance: The main application (main.py) continuously reads frames from the webcam, detects faces, compares each detected face against the stored encodings, and for every recognized person, automatically writes an attendance record (attendance_manager.py).

3.1 Sample Registration Screenshots

The screenshots below show the live registration process: OpenCV's Haar Cascade detector draws a green bounding box around the detected face in real time, confirming a face is present before a photo is captured.





4. Division of Work

The project was divided into two equal, largely independent halves so both members could work in parallel, connected by a shared interface agreed on before development began: a fixed folder structure (known_faces/<Name>), a shared encodings file (encodings.pickle), and a fixed attendance output format (Attendence.CSV with name, date and time columns).

Member 1 — Recognition Engine (Serena Salman)	Member 2 — Attendance & Main App (Lamia Gul)
register_faces.py — captures and saves face photos	attendance_manager.py — creates and updates Attendance.csv
encode_faces.py — generates encodings.pickle	main.py — runs the live webcam loop and ties recognition + attendance
face_recognizer.py — standalone recognition test script	README.md — setup and run documentation

5. Implementation Details — Member 2 (Attendance Logic & Main Application)

5.1 attendance_manager.py

- This module is responsible for all CSV read/write operations related to attendance.
- initialize_csv() — Creates Attendance.csv with the header row Name, Date, Time if the file does not already exist, so the application is self-sufficient on first run.
- is_already_marked_today(name, today_date) — Scans existing CSV rows to check whether a given person already has an entry for today's date — this is the core mechanism that prevents duplicate attendance records.
- mark_attendance(name) — Retrieves the current date and time, checks for an existing record via is_already_marked_today(), and appends a new row only if the person has not already been marked today. Returns True/False to indicate whether a new record was written.

5.2 main.py

This is the main driver script. It loads the shared encodings (produced by Member 1's encode_faces.py), opens the webcam, and runs a continuous loop that:

- Captures a frame and resizes it to 25% of its original size, to keep face recognition fast enough for real-time use.
- Converts the frame from BGR (OpenCV's default) to RGB (required by face_recognition).
- Detects all faces in the frame and computes a face encoding for each one.
- Compares each encoding against the known encodings using both compare_faces() and face_distance(), so that when a face is close to more than one known person, the single closest match is chosen rather than the first match above the threshold.
- Calls mark_attendance() the first time each person is recognized in the session, and tracks already-marked names in memory to avoid redundant CSV reads on every video frame.
- Draws a bounding box and name label on the original frame — green for recognized faces, red for unrecognized ('Unknown') faces — and displays the live annotated video feed.

6. Results

The completed system successfully detects faces in real time, distinguishes registered from unregistered individuals, and produces a clean attendance log. A sample of the generated Attendance.csv output is shown below:

Name	Date	Time
Serena Salman	2026-07-25	13:17:45
Lamia Gul	2026-07-25	13:42:10

Testing confirmed that: recognized individuals are correctly labeled with a green bounding box; unregistered individuals are labeled "Unknown" with a red bounding box and no CSV entry is created for them; and running the application multiple times on the same day does not produce duplicate rows for any individual.

7.Conclusion

This project successfully demonstrates an automated, contactless attendance system built with widely available open-source tools. Splitting the work into a recognition engine and an attendance/application layer, connected through a simple, agreed-upon file-based interface, allowed both team members to develop, test, and debug their components independently before a final integration pass — completing the project within the two-day timeframe.

Possible future improvements include storing attendance in a database instead of a CSV file, adding a simple GUI, and improving recognition robustness under varying lighting conditions